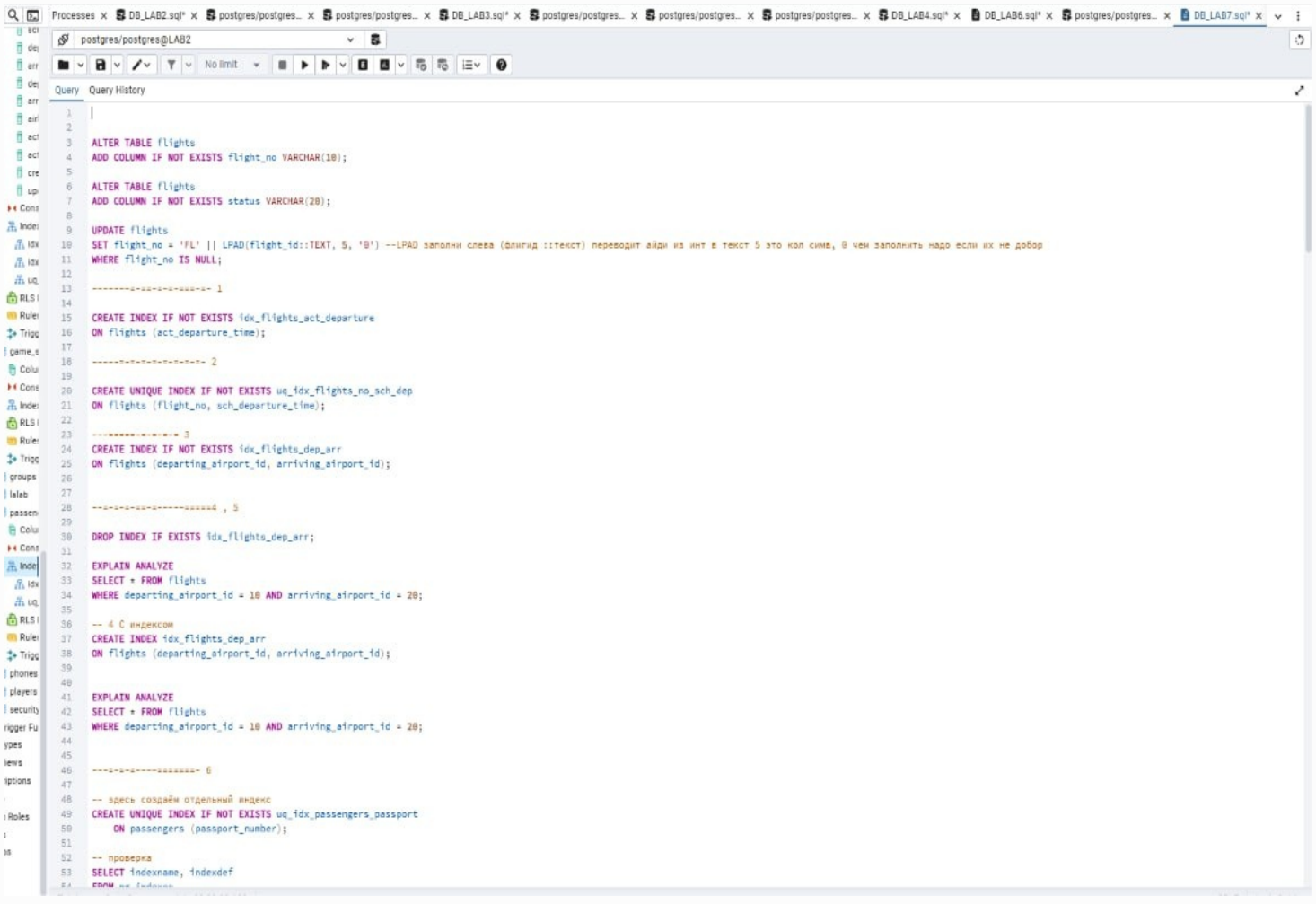


```
ALTER TABLE flights
ADD COLUMN IF NOT EXISTS flight_no VARCHAR(10);

ALTER TABLE flights
ADD COLUMN IF NOT EXISTS status VARCHAR(20);

UPDATE flights
SET flight_no = 'FL' || LPAD(flight_id::TEXT, 5, '0') --LPAD заполни слева (флигид ::текст) переводит айди из инт в текст 5 это кол симв, 0 чем заполнить надо если их не добор
WHERE flight_no IS NULL;

--------1-1-1-1-1---- 1

CREATE INDEX IF NOT EXISTS idx_flights_act_departure
ON flights (act_departure_time);

-----2-2-2-2-2-2-2-2- 2

CREATE UNIQUE INDEX IF NOT EXISTS uq_idx_flights_no_sch_dep
ON flights (flight_no, sch_departure_time);

---3------3-3-3-3-3 3
CREATE INDEX IF NOT EXISTS idx_flights_dep_arr
ON flights (departing_airport_id, arriving_airport_id);


---4-4-4-44-4-----44444 , 5

DROP INDEX IF EXISTS idx_flights_dep_arr;

EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departing_airport_id = 10 AND arriving_airport_id = 20;

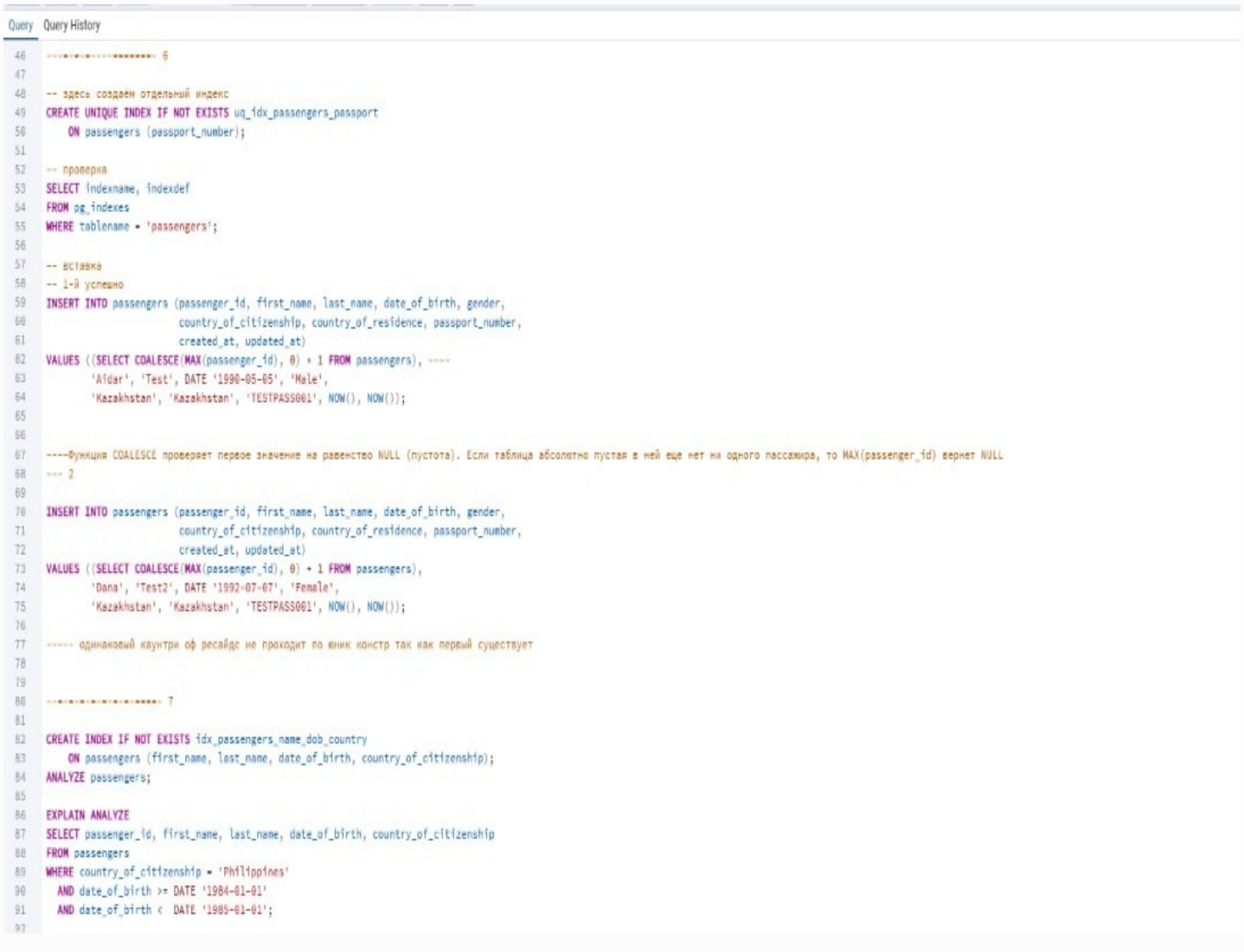
-- 4 С индексом
CREATE INDEX idx_flights_dep_arr
ON flights (departing_airport_id, arriving_airport_id);


EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departing_airport_id = 10 AND arriving_airport_id = 20;


---5-5-5----55555555- 6

-- здесь создаём отдельный индекс
CREATE UNIQUE INDEX IF NOT EXISTS uq_idx_passengers_passport
    ON passengers (passport_number);

-- проверка
SELECT indexname, indexdef
```



```
---6-6-6-66666666666- 6

-- здесь создаём отдельный индекс
CREATE UNIQUE INDEX IF NOT EXISTS uq_idx_passengers_passport
    ON passengers (passport_number);

-- проверка
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';

-- вставка
-- 1-й успешно
INSERT INTO passengers (passenger_id, first_name, last_name, date_of_birth, gender,
                        country_of_citizenship, country_of_residence, passport_number,
                        created_at, updated_at)
VALUES ((SELECT COALESCE(MAX(passenger_id), 0) + 1 FROM passengers), ----
        'Aidar', 'Test', DATE '1990-05-05', 'Male',
        'Kazakhstan', 'Kazakhstan', 'TESTPASS001', NOW(), NOW());


----Функция COALESCE проверяет первое значение на равенство NULL (пустота). Если таблица абсолютно пустая в ней еще нет ни одного пассажира, то MAX(passenger_id) вернет NULL
--- 2

INSERT INTO passengers (passenger_id, first_name, last_name, date_of_birth, gender,
                        country_of_citizenship, country_of_residence, passport_number,
                        created_at, updated_at)
VALUES ((SELECT COALESCE(MAX(passenger_id), 0) + 1 FROM passengers),
        'Dana', 'Test2', DATE '1992-07-07', 'Female',
        'Kazakhstan', 'Kazakhstan', 'TESTPASS001', NOW(), NOW());

----- одинаковый каунтри оф ресайдс не проходит по юник констр так как первый существует


---8-8-8-8-8-8-8-8-8-8- 7

CREATE INDEX IF NOT EXISTS idx_passengers_name_dob_country
    ON passengers (first_name, last_name, date_of_birth, country_of_citizenship);
ANALYZE passengers;

EXPLAIN ANALYZE
SELECT passenger_id, first_name, last_name, date_of_birth, country_of_citizenship
FROM passengers
WHERE country_of_citizenship = 'Philippines'
  AND date_of_birth >= DATE '1984-01-01'
  AND date_of_birth <  DATE '1985-01-01';
```

```sql
---------- 8


SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';


DROP INDEX IF EXISTS idx_passengers_name_dob_country;
DROP INDEX IF EXISTS uq_idx_passengers_passport;

--SELECT indexname FROM pg_indexes WHERE tablename = 'passengers';


----------  9

DROP INDEX IF EXISTS idx_booking_passenger_id;

EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;


CREATE INDEX idx_booking_passenger_id ON booking (passenger_id);
ANALYZE booking;

--ПОСЛЕ
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;


----------  10

CREATE INDEX IF NOT EXISTS idx_booking_flight_booking_flight
    ON booking_flight (booking_id, flight_id);
ANALYZE booking_flight;

EXPLAIN ANALYZE
SELECT f.flight_no, b.price
FROM booking b
JOIN booking_flight bf ON bf.booking_id = b.booking_id
JOIN flights f         ON f.flight_id   = bf.flight_id
WHERE b.price > 9000;




----------  11


UPDATE flights
SET status = CASE
        WHEN flight_id % 200 = 0 THEN 'Boarding'
        WHEN flight_id % 10  = 0 THEN 'Delayed'
        ELSE 'Departed'
    END
WHERE status IS NULL;

ANALYZE flights;

----------

CREATE INDEX IF NOT EXISTS idx_flights_status ON flights (status);
ANALYZE flights;

EXPLAIN ANALYZE
SELECT * FROM flights WHERE status = 'Departed';

EXPLAIN ANALYZE
SELECT * FROM flights WHERE status = 'Boarding';



select status, count(*) from flights
group by status

----------  12

CREATE INDEX IF NOT EXISTS idx_baggage_weight ON baggage (weight_in_kg);
ANALYZE baggage;

EXPLAIN ANALYZE
SELECT * FROM baggage WHERE weight_in_kg > 49;
EXPLAIN ANALYZE
SELECT * FROM baggage WHERE weight_in_kg > 5;

----------  13

CREATE INDEX IF NOT EXISTS idx_airport_name ON airport (airport_name);
--для LIKE по префиксу при не-C локали нужен класс операторов pattern_ops
CREATE INDEX IF NOT EXISTS idx_airport_name_pattern
ON airport (airport_name varchar_pattern_ops);
ANALYZE airport;


EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE 'A%';
EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE '%Airport';
```

```
---xx-x---x-x-xx-x---- 14


CREATE INDEX IF NOT EXISTS idx_airport_country_city ON airport (country, city);
ANALYZE airport;

EXPLAIN ANALYZE SELECT * FROM airport WHERE country = 'Kazakhstan';  -- по country
EXPLAIN ANALYZE SELECT * FROM airport WHERE city = 'Astana'; -- по city
EXPLAIN ANALYZE SELECT * FROM airport WHERE country = 'Kazakhstan' AND city = 'Astana';  -- по обоим

---x-x-x-x-x-x-x-x-x-6-x-x-7-x- 15

CREATE INDEX IF NOT EXISTS idx_passengers_lower_last_name
ON passengers (LOWER(last_name));
ANALYZE passengers;

EXPLAIN ANALYZE
SELECT * FROM passengers WHERE LOWER(last_name) = 'irnis';
EXPLAIN ANALYZE
SELECT * FROM passengers WHERE last_name = 'Irnis';


---x-xx-x-x-x-x-xx-x-x-x-x- 16


CREATE INDEX IF NOT EXISTS idx_booking_passenger_price
ON booking (passenger_id, price);
VACUUM ANALYZE booking; -- обновляет visibility map, нужно для Index Only Scan

EXPLAIN ANALYZE
SELECT price FROM booking WHERE passenger_id = 100;
```

```
---x-xx-x-x-x-x-x-xx-x-xx-x- 16


CREATE INDEX IF NOT EXISTS idx_booking_passenger_price
ON booking (passenger_id, price);
VACUUM ANALYZE booking; -- обновляет visibility map, нужно для Index Only Scan

EXPLAIN ANALYZE
SELECT price FROM booking WHERE passenger_id = 100;

------x-x-xxx-x-x-x-x-x-x-x- 17

CREATE UNIQUE INDEX uq_idx_airline_code ON airline (airline_code);


SELECT airline_code, COUNT(*)
FROM airline
GROUP BY airline_code
HAVING COUNT(*) > 1;


SELECT *
FROM airline
WHERE airline_code IN (SELECT airline_code
                       FROM airline
                       GROUP BY airline_code
                       HAVING COUNT(*) > 1)
ORDER BY airline_code, airline_id;



UPDATE airline
SET airline_code = LEFT(airline_code, 20) || '_' || airline_id, updated_at = NOW()
WHERE airline_id IN (
    SELECT airline_id
    FROM (SELECT airline_id,
                 ROW_NUMBER() OVER (PARTITION BY airline_code ORDER BY airline_id) AS rn
          FROM airline) t
    WHERE rn > 1
);


---ROW_NUMBER() - это функция, которая просто нумерует строки подряд (1, 2, 3...)

---PARTITION BY airline_code - говорит: «Нумеруй строки не подряд по всей таблице, а отдельно для каждого уникального кода авиакомпании».

---ORDER BY airline_id - внутри каждой группы строки сортируются по ID (самый первый созданный получает номер 1, следующий - 2 и т.д.).

---Результат: База как бы расставляет порядковые номера (rn - row number) для дубликатов: у первой нормальной строки будет rn = 1, а у всех последующих дубликатов с тем же кодом будет rn = 2, rn = 3 и т.д.


--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

-- Для всех найденных дубликатов (у которых rn > 1) мы меняем airline_code
```